SRH UNIVERSITY

APPLIED MECHATRONICS

100007801_Aly_AP

Automated Pencil Production Line

Advanced Programming Final Project

Mohannad Aly

Student ID: 100007801

Course: Advanced Programming

Instructor: Esteban Pozo

Due date: June 20, 2026

A Python · InfluxDB · Grafana · Docker implementation

Abstract

This project implements a software simulation of an automated pencil manufacturing line. The production sequence contains six stations: wooden-body feeding, graphite-core insertion, ferrule installation, eraser insertion, final quality inspection, and sorting. A Python state machine controls the sequence, creates simulated sensor values, records production statistics, and separates product-quality defects from equipment faults. The browser-based human-machine interface provides Start, Stop, Reset, Acknowledge, Emergency Stop, speed, and defect-probability controls. It also displays the current station, machine state, temperature, cycle time, first-pass yield, recent products, rejection reasons, and the event log. Production telemetry is written asynchronously to InfluxDB by using line protocol and is visualized through an automatically provisioned Grafana dashboard. Docker Compose packages the Python service, database, and dashboard as one reproducible system with persistent volumes. Automated unit and API tests verify the core controls, quality logic, fault handling, and web endpoints. The result demonstrates how programming, databases, visualization, version control, containerization, and artificial-intelligence-assisted development can be combined in a small manufacturing monitoring application.

Keywords: *Python, manufacturing simulation, HMI, InfluxDB, Grafana, Docker, quality control*

Submission package: The repository includes the source code, Docker configuration, Grafana provisioning, automated tests, GitHub Pages website, editable report, and final PDF.

Contents

Document structure: The main technical documentation occupies pages 4-10. References and the required AI-use annex follow separately.

Introduction

Modern manufacturing software combines control logic, operator interaction, data collection, and visualization. This project applies those ideas to an educational production-line simulation. The objective is not to control real machinery; it is to demonstrate a clear software architecture in which a state machine produces repeatable process behavior, an HMI allows operator control, and time-series data supports monitoring and analysis.

The implementation uses only Python standard-library modules for the runtime web server, threading, queues, and simulation logic. Python provides threaded HTTP-server classes and concurrency tools suitable for educational network and I/O tasks, while its pseudo-random module supports repeatable simulated measurements when a seed is supplied (Python Software Foundation, 2026a, 2026b, 2026c). The project therefore remains easy to inspect and can be previewed even before the database containers are started.

Task 1 - Product and Production-Line Concept	10 points

The selected product is a conventional wooden pencil with an eraser. It was chosen because the product has distinct mechanical components and measurable assembly characteristics. The simulated line contains more than the required four stages and models both assembly and inspection.

Wooden body feed: a pencil body is released from the feeder and checked for presence and damage.
Graphite core insertion: the core is inserted and checked for breakage and lateral misalignment.
Ferrule installation: the metal eraser holder is pressed onto the body and clamp force is evaluated.
Eraser insertion: the eraser is inserted into the ferrule and its presence and depth are checked.
Final quality inspection: overall length and surface quality are evaluated.
Sorting and discharge: accepted products and rejected products are routed to separate outputs.

Each pencil receives a product identifier and sensor-like measurements. A product that fails any quality checkpoint continues through the educational sequence but is marked for rejection at the sorting station. The rejection message identifies the exact reason, such as “Graphite core misaligned” or “Eraser missing.” Equipment faults are handled separately because a faulty machine should stop production, while a defective product should be counted and removed without stopping the line.

4

Task 2 - Program Development

60 points

2.1 Software Architecture

The system is divided into the browser HMI, Python application, InfluxDB, and Grafana. Python contains the production model, JSON control API, and asynchronous database writer. Docker Compose starts the services, defines persistent storage, and uses health-based startup dependencies (Docker, 2026).

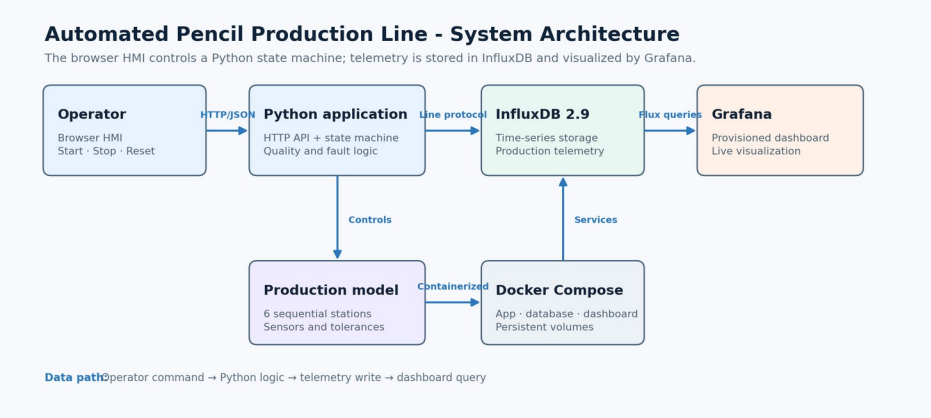

Figure 1. System architecture and data flow (author's illustration).

The ProductionLine class protects its machine snapshot with a re-entrant lock. A production thread waits for Start and executes the stage sequence. Stop pauses the cycle, Reset clears counters while stopped, Acknowledge clears a fault, and Emergency Stop changes the state immediately. The HMI uses endpoints such as /api/status, /api/start, /api/stop, and /api/settings.

2.2 Quality-Control Logic

The configured final-product defect probability is distributed across five checkpoints. To avoid multiplying the intended rejection rate, the program calculates a per-stage probability of 1 - (1 - p)^(1/5). A failed measurement is changed to an out-of-tolerance value and its reason is preserved until sorting.

Station	Example measurement	Possible rejection reason
Body feed	Body length / presence	Cracked body or missing body
Core insertion	Graphite offset	Misaligned or broken graphite
Ferrule press	Clamp force	Ferrule not seated / low force
Eraser insertion	Presence / depth	Missing or incorrect insertion depth
Final inspection	Length / surface score	Length or visual surface defect

5

2.3 Human-Machine Interface

The browser HMI uses HTML, CSS, and JavaScript. It includes Start, Stop, Reset, Acknowledge, Emergency Stop, speed, defect probability, and a demonstration-fault button. The page polls the Python API every 500 ms and updates the active stage, counters, yield, temperature, cycle time, database status, recent products, and event log.

Figure 2. Browser HMI during production (author's screenshot).

Color and text are combined so state is not communicated by color alone. A fault changes the state indicator, displays an alarm banner, blocks Start, and requires acknowledgement. Emergency Stop retains a distinct state. Product defects only update the product record and rejected counter; they do not stop the machine.

2.4 Backend Robustness

The database client uses a bounded queue and separate worker thread, preventing network delays from blocking production. Writes have a timeout, one retry, and health checks. API bodies are size-limited and validated, and dynamic HMI text is HTML-escaped. Real deployment would still require authentication and a production-grade server.

6

2.5 InfluxDB Database and Grafana Dashboard

InfluxDB stores timestamped telemetry in the pencil_production bucket. Line-protocol fields include temperature, total/good/defect counts, cycle time, state code, stage index, defective flag, yield, machine-state text, and defect reason. Tags identify the machine, state, stage, and event. Docker setup and HTTP writes follow the InfluxDB v2 documentation (InfluxData, 2026a, 2026b).

Grafana is provisioned from YAML and JSON files. Its Flux data source receives the organization, bucket, and token through environment variables. The dashboard shows temperature, cycle time, output, first-pass yield, mapped machine state, and latest status. Configuration-file provisioning makes the setup reproducible (Grafana Labs, 2026).

Figure 3. Configured Grafana dashboard layout using representative project telemetry (author's own visualization).

Docker services: HMI: 8000; InfluxDB: 8086; Grafana: 3000. Named volumes preserve data between restarts.

7

2.6 Testing and Demonstration Results

The project contains automated tests based on Python's unittest framework. The tests set a short cycle time and disable the external database so that logic can be verified quickly and deterministically. Eight tests were executed successfully during preparation of the submission package.

Test area	Expected behavior	Result
Start and production	Product completes; 0% defects gives only good output	Pass
Stop and reset	Stop changes state; reset clears counters	Pass
Emergency stop	Restart is blocked until acknowledgement	Pass
Settings validation	Invalid speed/defect values are rejected	Pass
Injected fault	Line enters FAULT with its message	Pass
Status API	JSON contains state and temperature	Pass
Start API	POST request starts the production line	Pass
Static HMI	Root page serves the interface	Pass

A demonstration produced accepted and rejected pencils with explicit reasons. The line remained responsive with InfluxDB disabled because telemetry is decoupled from the production thread. With Docker running, the same fields are stored and queried by Grafana.

2.7 Program Files

File or folder	Purpose
app.py	Starts the production model and threaded HTTP server.
production_line/engine.py	State machine, stages, defects, faults, counters, and telemetry.
influx_writer.py	Background health checks and line-protocol writes.
static/	Browser HMI and API client.
grafana/	Provisioned data source and dashboard.
tests/ and Compose	Automated tests and three-service deployment.

8

Task 3 - Development Tools and Website

20 points

Git and GitHub are used for version control, repository hosting, and project publishing. The repository separates source code, tests, infrastructure configuration, website files, and documentation so that changes can be reviewed logically. The included GitHub Actions workflow uploads the website folder and deploys it to GitHub Pages. GitHub Pages can publish from a branch or a custom Actions workflow; this project uses the workflow approach (GitHub, 2026).

Docker packages the Python application, InfluxDB, and Grafana into isolated services. The Dockerfile runs the application as a non-root user and includes an HMI health check. Docker Compose supplies credentials through environment variables, creates named volumes, waits for InfluxDB health before starting dependent services, and provides a one-command setup with docker compose up --build.

ChatGPT was used to support planning, code generation, review, test design, documentation, and visual consistency. OpenAI describes ChatGPT as an assistant that can support coding, writing, studying, and analysis tasks (OpenAI, 2026). All generated material was reviewed. Specific corrections included removing an unnecessary third-party framework, correcting the probability calculation, moving database writes to a background queue, adding HTML escaping and input validation, and clearly documenting security limitations. The relevant prompts, outputs, and corrections are included in the annex.

Project website: https://mohannad275.github.io/100007801_Aly_AP/

Repository: https://github.com/mohannad275/100007801_Aly_AP

Website content: The static site contains four explanatory paragraphs and two project images, meeting the assignment limit while providing direct links to the repository.

9

Task 4 - Conclusion and Further Development

10 points

The project meets the functional scope: more than four manufacturing stages, Python backend logic, explicit defect reasons, required HMI controls, InfluxDB telemetry, and a Grafana dashboard. Its strongest feature is separation of concerns: assembly, machine state, controls, database communication, visualization, deployment, testing, and publishing are organized in clear modules.

The HMI demonstrates the live sequence, adjustable parameters, accepted and rejected products, and fault recovery. A background database writer prevents telemetry problems from freezing production, while automated tests verify essential commands and endpoints.

The main weakness is that this is a simulation, not a real-time controller. It has no deterministic timing, PLC interface, physical safety circuit, or certified emergency stop; measurements and faults are statistical. The lightweight HTTP server is not suitable for public production use. A deployed system would require a hardened server, encryption, authentication, authorization, network segmentation, secure secrets, audit retention, backups, and maintenance procedures.

Further development could connect OPC UA or a PLC, and add recipes, maintenance work orders, alarm history, OEE, retention policies, Grafana alerts, and predictive maintenance. Real OT systems must prioritize safety, availability, integrity, and controlled change; NIST emphasizes their unique reliability, performance, and safety requirements (Stouffer et al., 2023). The application is therefore an educational prototype, not a controller for physical machinery.

Overall assessment: The solution is complete for the academic brief, reproducible through Docker, transparent about AI use, and structured so that future PLC, SCADA, CMMS, and advanced quality modules can be added.

10

References

Docker, Inc. (2026). Control startup and shutdown order in Compose. https://docs.docker.com/compose/how-tos/startup-order/

GitHub, Inc. (2026). Configuring a publishing source for your GitHub Pages site. https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

Grafana Labs. (2026). Configure the InfluxDB data source. https://grafana.com/docs/grafana/latest/datasources/influxdb/configure/

InfluxData. (2026a). InfluxDB HTTP API. https://docs.influxdata.com/influxdb/v2/api/

InfluxData. (2026b). Install InfluxDB using Docker Compose. https://docs.influxdata.com/influxdb/v2/install/use-docker-compose/

OpenAI. (2026). What is ChatGPT: FAQ. https://help.openai.com/en/articles/12677804

Python Software Foundation. (2026a). http.server - HTTP servers. https://docs.python.org/3/library/http.server.html

Python Software Foundation. (2026b). random - Generate pseudo-random numbers. https://docs.python.org/3/library/random.html

Python Software Foundation. (2026c). threading - Thread-based parallelism. https://docs.python.org/3/library/threading.html

Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., & Thompson, M. (2023). Guide to operational technology (OT) security (NIST SP 800-82 Rev. 3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-82r3

Annex - AI Prompts, Outputs, and Corrections

The following prompts and outputs document the use of ChatGPT in accordance with the project brief. The output excerpts are condensed to the portions that directly influenced the submitted implementation.

Prompt 1 - Overall Architecture

> **Prompt:** Design a complete Python simulation of an automated pencil production line for an Advanced Programming project. It must include at least four assembly stages, explicit product-defect reasons, Start/Stop/Reset controls, an HMI, InfluxDB, Grafana, Docker, automated tests, and a clear separation between product defects and machine faults.

Relevant AI output:

```
Operator browser → Python HTTP API → ProductionLine state machine
                                          ↓
                                   InfluxDB v2
                                          ↓
                                     Grafana

States: IDLE, RUNNING, STOPPED, FAULT, EMERGENCY_STOP
Stages: body, graphite, ferrule, eraser, inspection, sorting
```

Correction and validation: The initial proposal used a third-party web framework. It was replaced by ThreadingHTTPServer to remove runtime dependencies. The InfluxDB writer was moved to a separate queue and thread so that a database delay cannot stop production.

Prompt 2 - Quality-Control Rules

> **Prompt:** Generate realistic quality-control measurements and defect reasons for body feeding, graphite insertion, ferrule pressing, eraser insertion, and final pencil inspection.

Relevant AI output: body length and presence, graphite offset and integrity, ferrule force, eraser depth and presence, final length, and visual surface score.

Correction and validation: Applying the configured defect probability at all five stations caused the final defect rate to be much too high. The probability was converted to a per-stage value using 1 - (1 - p)^(1/5), and automated testing was run with a zero-defect setting.

12

AI Output Review and Benefits

Prompt 3 - HMI, InfluxDB, and Grafana

> **Prompt:** Create an industrial browser HMI with Start, Stop, Reset, Acknowledge, Emergency Stop, live process stages, production statistics, fault display, event log, recent product table, and settings. Add InfluxDB line-protocol fields and a provisioned Grafana dashboard.

Relevant AI output:

```
production_metrics,machine=pencil_line,state=RUNNING,stage=Ferrule_installation
  temperature=37.8,total_count=148i,good_count=137i,defect_count=11i,
  cycle_time=5.36,state_code=1i,yield_percent=92.6
```

Correction and validation: API input limits, numeric validation, HTML escaping, database timeouts, queue limits, health checks, and one retry were added. The HMI was run locally and the source was tested through its API endpoints.

Prompt 4 - Testing and Documentation

> **Prompt:** Write automated tests and a submission-ready report explaining the product, implementation, tools, weaknesses, AI use, and future improvements.

Relevant AI output: an eight-test plan, the Task 1-4 report structure, Docker demonstration steps, GitHub Pages content, and an AI correction log.

Correction and validation: All eight tests were executed successfully. The Word document was rendered to PDF, each page was visually inspected, and layout problems were corrected before delivery.

Benefits of ChatGPT

ChatGPT accelerated brainstorming, scaffolding, test-case generation, documentation organization, and consistency checking across Python, JavaScript, Docker, InfluxDB, Grafana, the website, and the report. It was particularly useful for producing alternative designs and identifying edge cases. Human review remained essential because generated code can contain incorrect assumptions, excessive dependencies, unsafe defaults, probability mistakes, or deployment claims that have not been tested.

> **Final disclosure:** ChatGPT was used as a development assistant. The submitted code and documentation were reviewed, tested, corrected, and assembled for this project. AI-generated material was not accepted without validation.

13